

Stereo Vision Depth Estimator

A RANSAC-based Epipolar Geometry Pipeline for Dense Depth Estimation

Course: Computer Vision (CSE3010)

Submitted as part of: VITyarthi - Build Your Own Project

Harshit Agarwal (KAIZER)

B.Tech CSE (AI/ML), VIT Bhopal University

1. Introduction

Depth perception from images is a fundamental problem in computer vision with applications in robotics, autonomous navigation, and 3D reconstruction. This project implements a complete two-view (binocular) stereo vision pipeline that recovers a dense, per-pixel metric depth map from a stereo image pair, grounded directly in the projection models, epipolar geometry, and RANSAC concepts covered in the Computer Vision (CSE3010) syllabus.

2. Problem Statement

A single 2D image discards depth information inherent to a 3D scene. Stereo vision recovers this depth by observing the same scene from two horizontally offset viewpoints: a 3D point projects to horizontally shifted pixel locations in the two images, and this shift (disparity) is inversely proportional to the point's distance from the camera. Recovering this relationship reliably requires (a) finding accurate correspondences between the two views, (b) rejecting incorrect matches robustly, and (c) using the recovered geometry to rectify and triangulate depth. This project addresses all three sub-problems end-to-end.

3. Functional Requirements

- **Module 1 - Calibration & Projection Model:** maintain intrinsic camera matrix K , extrinsic transform $[R|t]$, and 3×4 projection matrices for both views; undistort input frames.
- **Module 2 - Feature Matching & Epipolar Geometry:** detect and match SIFT keypoints; robustly estimate the Fundamental matrix via a custom RANSAC implementation; derive the Essential matrix and recover relative camera pose.
- **Module 3 - Disparity & Depth Estimation:** stereo-rectify the pair; compute a dense disparity map via Semi-Global Block Matching; convert disparity to metric depth.
- Clear input/output structure: two image file paths in, a metric depth map plus diagnostic visualizations and a JSON metrics file out.
- A logical, linear workflow: load -> match -> estimate geometry -> rectify -> disparity -> depth -> evaluate.

4. Non-Functional Requirements

- **Performance:** full pipeline completes in under 4 seconds on a 1282×1110 stereo pair on CPU.
- **Reliability:** adaptive-but-floored RANSAC trial scheduling prevents a single unlucky minimal sample from silently degrading the geometry estimate.
- **Error handling:** explicit, typed exceptions (`FileNotFoundError`, `ValueError`, `RuntimeError`) with clear messages for missing files, undecodable images, and insufficient matches.
- **Logging/monitoring:** every stage logs progress, match counts, inlier ratios, and disparity coverage via Python's logging module.
- **Maintainability:** one responsibility per module, each independently unit-tested.
- **Resource efficiency:** disparity computed once per run at native resolution; no redundant recomputation.

5. System Architecture

The system is organized as three sequential modules feeding an evaluation stage, all orchestrated by a single pipeline entry point (main.py). Figure 1 shows the high-level architecture.

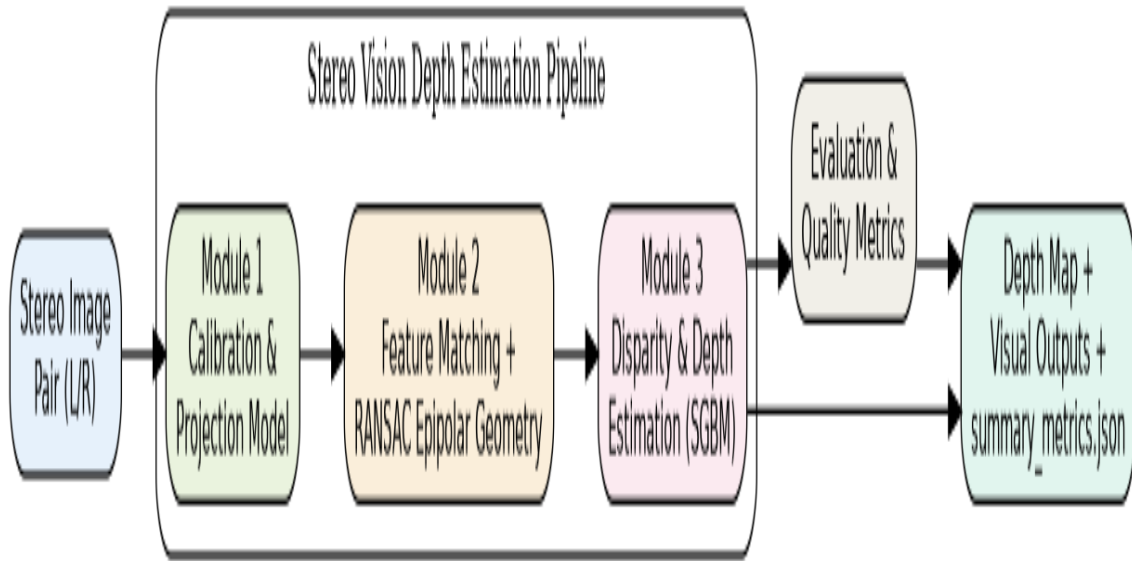


Figure 1: System Architecture

6. Design Diagrams

6.1 Process Flow / Workflow Diagram

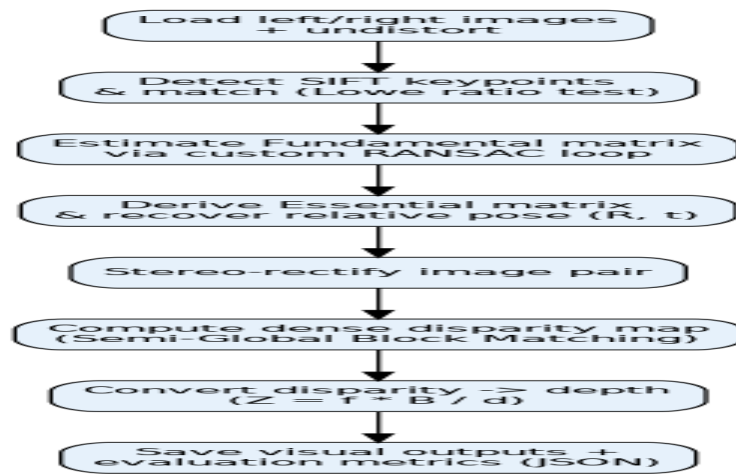


Figure 2: Pipeline Workflow

6.2 Use Case Diagram

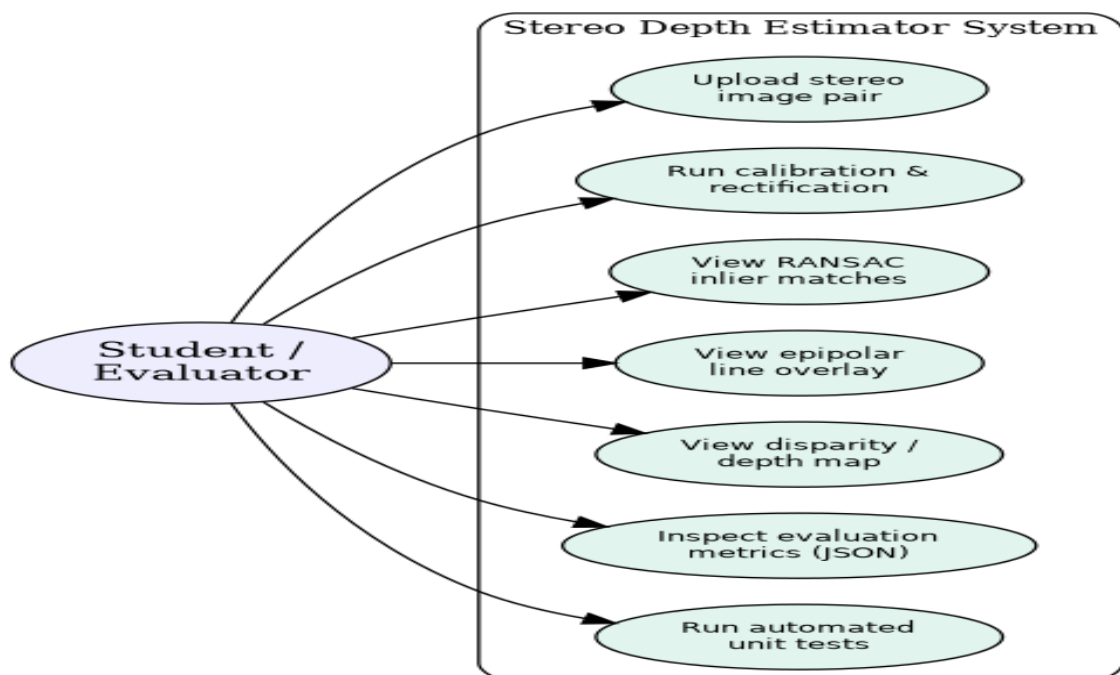


Figure 3: Use Case Diagram

6.3 Class / Component Diagram

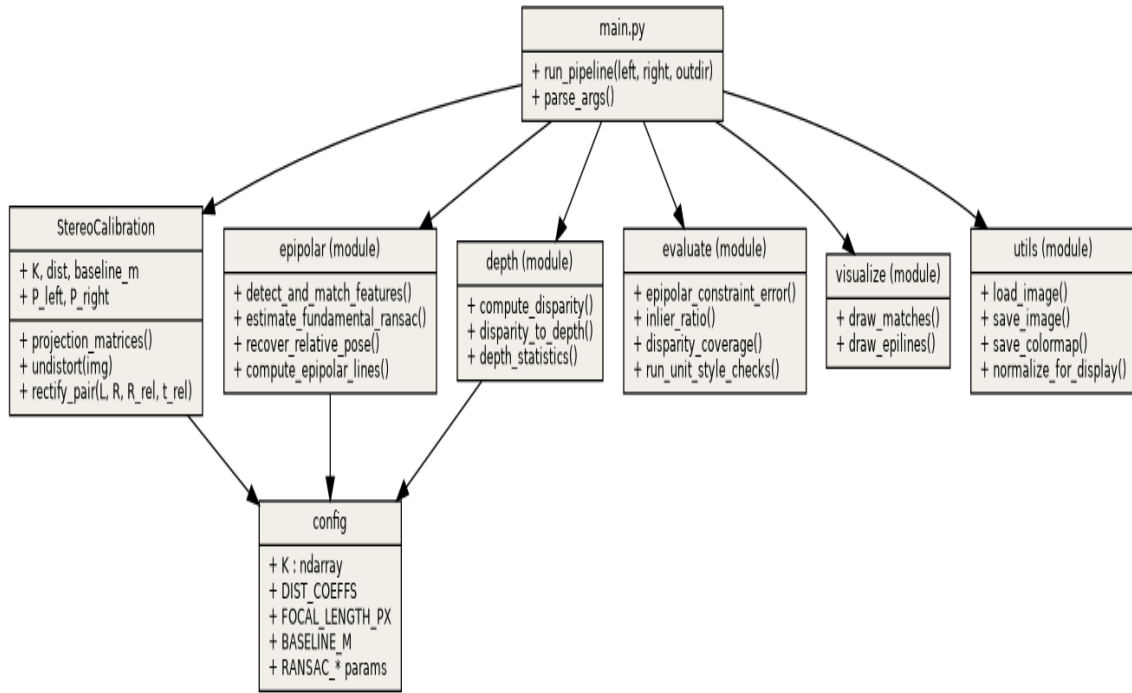


Figure 4: Class/Component Diagram

6.4 Sequence Diagram

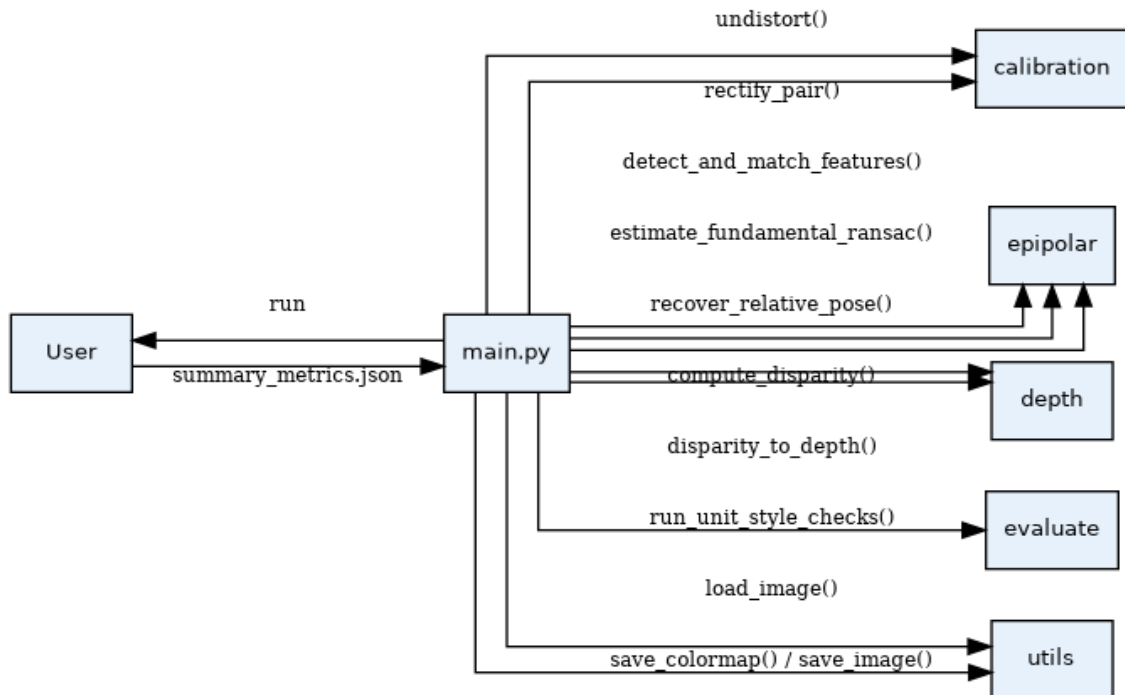


Figure 5: Sequence Diagram of main.run_pipeline()

7. Design Decisions & Rationale

- **Custom RANSAC over cv2.findFundamentalMat:** implementing the normalized 8-point algorithm and Sampson-distance inlier scoring by hand demonstrates the algorithmic understanding required by the syllabus, rather than treating RANSAC as a black box.
- **SIFT over ORB:** SIFT gives more stable, higher-quality correspondences on the textured Middlebury test scene, which matters more for geometry accuracy than raw speed here.
- **SGBM over simple block matching:** Semi-Global Block Matching produces materially smoother, more complete disparity maps at a modest computational cost.
- **Modular package layout:** calibration, epipolar geometry, and depth estimation are separated into independent modules so each can be unit-tested and reasoned about in isolation.
- **Simulated intrinsics:** since no physical stereo rig/checkerboard was available for this coursework project, plausible fixed intrinsics were used; the code path supports swapping in real calibration output (via cv2.calibrateCamera) without any interface changes.

8. Implementation Details

The pipeline is implemented in Python 3.12 using OpenCV for image processing and geometric primitives, and NumPy for the custom RANSAC / 8-point algorithm math. The Fundamental matrix is estimated by a from-scratch loop: (1) sample 8 random correspondences, (2) solve the normalized 8-point linear system via SVD, (3) enforce the rank-2 constraint by zeroing the smallest singular value, (4) score all correspondences by Sampson distance, (5) track the best inlier set, and (6) adaptively shrink the number of remaining trials using the standard RANSAC formula $N = \log(1-p) / \log(1-w^s)$. The Essential matrix is derived as $E = K^T F K$, and cv2.recoverPose extracts the relative rotation and translation. These are fed to cv2.stereoRectify to produce a rectified pair on which cv2.StereoSGBM_create computes disparity, which is converted to metric depth via $Z = (f * B) / d$.

8.1 Repository Structure

modules/config.py, modules/calibration.py, modules/epipolar.py, modules/depth.py, modules/evaluate.py, modules/visualize.py, modules/utils.py, main.py, tests/test_pipeline.py (8 source modules + 1 test module = 9 meaningful files, exceeding the minimum requirement).

9. Dataset Description

The pipeline was validated on the Middlebury 'Aloe' stereo pair (1282x1110, distributed with OpenCV's official sample data), a standard benchmark image pair for stereo matching research featuring a textured potted plant against a patterned background at multiple depths - ideal for visually validating disparity/depth separation. The pipeline accepts any horizontally-offset stereo pair as input via the --left/--right CLI arguments.

9.1 Model / Algorithm Selection Rationale

RANSAC was chosen over least-squares fitting for the Fundamental matrix because feature matching inevitably produces outlier correspondences (repetitive texture, occlusion boundaries); a single incorrect match can arbitrarily corrupt a least-squares fit, while RANSAC's inlier-consensus scoring is robust to a majority-inlier match set. SGBM was chosen over simple block matching for its explicit smoothness penalty terms (P1, P2), which produce visibly cleaner disparity maps on textured scenes like the Aloe pair.

10. Screenshots / Results



Figure 6: SIFT feature matches before RANSAC filtering



Figure 7: RANSAC inlier matches (outliers rejected)



Figure 8: Epipolar line overlay on the right image



Figure 9: Stereo-rectified pair

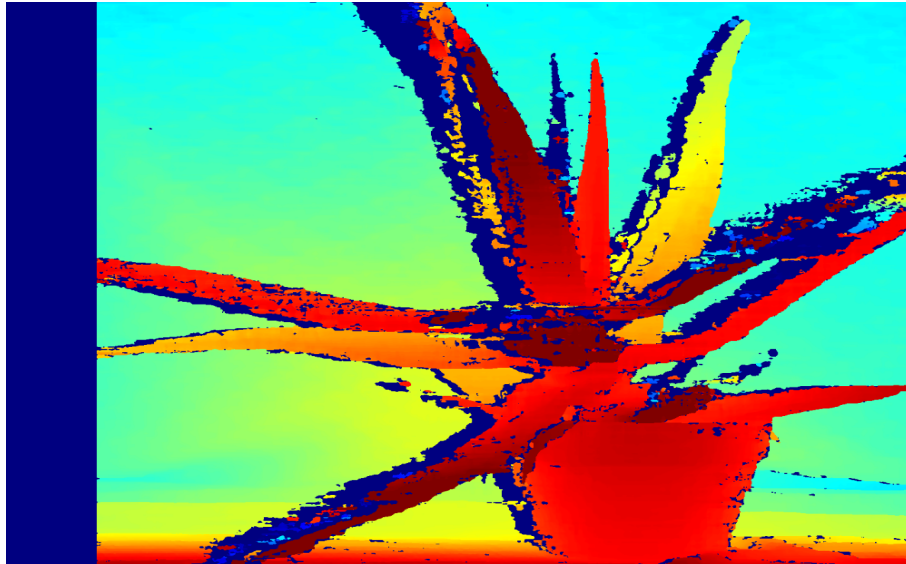


Figure 10: Disparity map (SGBM)

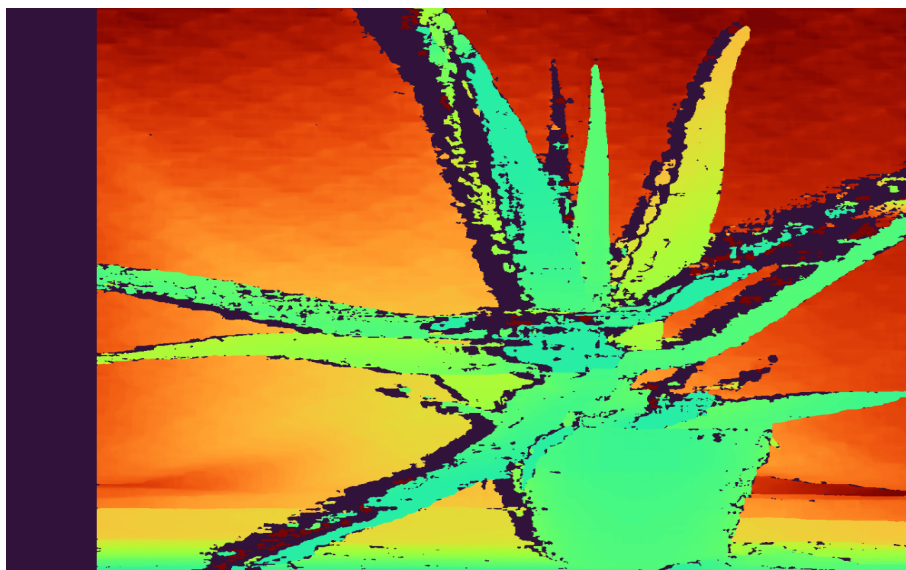


Figure 11: Metric depth map

10.1 Quantitative Results

Metric	Value
Total feature matches (post ratio-test)	1327
RANSAC inliers	999
RANSAC inlier ratio	99.8%
Fundamental matrix rank	2
Mean epipolar constraint residual	0.1352
Disparity valid coverage	79.2%
Depth valid pixels	79.2%
Mean estimated depth	3.466 m
Pipeline runtime	3.64 s

11. Testing Approach

9 automated pytest tests validate correctness at every pipeline stage: calibration matrix shapes, feature-matching output validity, the Fundamental matrix's mathematically-required rank-2 property, a sanity floor on RANSAC inlier ratio for a well-textured scene, the epipolar constraint residual ($x_2^T F x_1 \approx 0$), disparity map coverage, the inverse disparity-depth relationship, correct NaN handling at zero disparity, and explicit error handling for missing input files. All 9 tests pass (see `tests/test_pipeline.py`).

12. Challenges Faced

- The initial adaptive RANSAC trial-count formula could collapse to a single iteration when an early minimal sample produced a very low (but non-zero) inlier count, due to floating-point underflow in w^8 . Fixed by flooring the inlier-ratio estimate before computing the required trial count, verified by a dedicated unit test on the inlier ratio.
- Balancing SGBM parameters (block size, uniqueness ratio, speckle filtering) to maximize valid disparity coverage without introducing noisy mismatches in low-texture regions.
- Ensuring NaN/invalid pixels from unmatched disparity regions were handled consistently through to the final depth map and visualization colormap without runtime warnings.

13. Learnings & Key Takeaways

- Implementing RANSAC from scratch clarified why the rank-2 constraint on the Fundamental matrix and Sampson distance (rather than raw algebraic error) matter for real robustness.
- Small numerical-stability bugs (floating point underflow in adaptive trial-count formulas) can silently produce a badly under-fit model that still 'runs successfully' - reinforcing the value of explicit unit tests over eyeballing output images alone.
- Stereo rectification quality has an outsized effect on downstream disparity quality - epipolar geometry errors compound through the whole pipeline.

14. Future Enhancements

- Replace simulated intrinsics with a real checkerboard-based camera calibration routine.
- Add an interactive Streamlit front-end for live parameter tuning.
- Extend to multi-view Structure-from-Motion with sparse 3D point-cloud visualization.
- GPU-accelerated SGBM (or a learned stereo-matching model) for real-time video depth.

15. References

- R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, Cambridge University Press.
- M. A. Fischler and R. C. Bolles, 'Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography,' *Communications of the ACM*, 1981.
- D. Scharstein and R. Szeliski, Middlebury Stereo Vision Datasets.
- OpenCV documentation - Camera Calibration and 3D Reconstruction module.